Module 11

SQL Examples

Milav Dabgar

www.milav.in

March 17, 2026

Week Recap

- Basic notions of Relational Database Models

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance
- Keys, primary as well as foreign

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance
- Keys, primary as well as foreign
- Relational algebra with operators

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance
- Keys, primary as well as foreign
- Relational algebra with operators
- Relational query language

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance
- Keys, primary as well as foreign
- Relational algebra with operators
- Relational query language
- DDL (Data Definition)

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance
- Keys, primary as well as foreign
- Relational algebra with operators
- Relational query language
- DDL (Data Definition)
- DML (Basic Query Structure)

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance
- Keys, primary as well as foreign
- Relational algebra with operators
- Relational query language
- DDL (Data Definition)
- DML (Basic Query Structure)
- Detailed understanding of basic query structure

Week Recap

- Basic notions of Relational Database Models
- Attributes and their types
- Mathematical structure of relational model
- Schema and Instance
- Keys, primary as well as foreign
- Relational algebra with operators
- Relational query language
- DDL (Data Definition)
- DML (Basic Query Structure)
- Detailed understanding of basic query structure
- Set operations, null values, and aggregation

Module Objectives

Goals

- To recap various basic SQL features through example workout

Module Outline

- Examples of basic SQL

SELECT

Select distinct

Example

- From the **classroom** relation in the figure, find the names of buildings in which every individual classroom has capacity less than 100 (removing the duplicates).

classroom relation

building	room_number	capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Query:

```
select distinct building
from classroom
where capacity < 100;
```

Output:

building
Painter
Taylor
Watson

Select all

Example

- From the **classroom** relation in the figure, find the names of buildings in which every individual classroom has capacity less than 100 (without removing the duplicates).

classroom relation

building	room_number	capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Query:

```
select all building
from classroom
where capacity < 100;
```

Output:

building
Painter
Taylor
Watson
Watson

Cartesian Product / AS

Cartesian Product

Overview

- Find the list of all students of departments which have a budget ¡ $0.1 million

```
select name, budget
from student, department
where student.dept_name = department.dept_name
  and budget < 100000;
```

Cartesian Product

Overview

- Find the list of all students of departments which have a budget ¡ $0.1 million

```
select name, budget
from student, department
where student.dept_name = department.dept_name
  and budget < 100000;
```

- The above query first generates every possible student-department pair, which is the Cartesian product of `student` and `department`. Then, it filters all the rows with `student.dept_name = department.dept_name` and `budget < 100000`.

Cartesian Product

Overview

- Find the list of all students of departments which have a budget ¡ $0.1 million

```
select name, budget
from student, department
where student.dept_name = department.dept_name
  and budget < 100000;
```

- The above query first generates every possible student-department pair, which is the Cartesian product of `student` and `department`. Then, it filters all the rows with `student.dept_name = department.dept_name` and `budget < 100000`.
- The common attribute `dept_name` in the resulting table are renamed using the relation name (`student.dept_name` and `department.dept_name`)

Rename AS Operation

Query

- The same query in the previous slide can be framed by renaming the tables as shown below.

```
select S.name as studentname, budget as deptbudget
from student as S, department as D
where S.dept_name = D.dept_name
  and budget < 100000;
```

Output:

studentname	deptbudget
Brandt	50000

Rename AS Operation

Query

- The same query in the previous slide can be framed by renaming the tables as shown below.

```
select S.name as studentname, budget as deptbudget
from student as S, department as D
where S.dept_name = D.dept_name
  and budget < 100000;
```

- The above query renames the relation `student` as `S` and the relation `department` as `D`

Output:

studentname	deptbudget
Brandt	50000

Rename AS Operation

Query

- The same query in the previous slide can be framed by renaming the tables as shown below.

```
select S.name as studentname, budget as deptbudget
from student as S, department as D
where S.dept_name = D.dept_name
  and budget < 100000;
```

- The above query renames the relation `student` as `S` and the relation `department` as `D`
- It also displays the attribute `name` as `StudentName` and `budget` as `DeptBudget`.

Output:

studentname	deptbudget
Brandt	50000

Rename AS Operation

Query

- The same query in the previous slide can be framed by renaming the tables as shown below.

```
select S.name as studentname, budget as deptbudget
from student as S, department as D
where S.dept_name = D.dept_name
  and budget < 100000;
```

- The above query renames the relation `student` as `S` and the relation `department` as `D`
- It also displays the attribute `name` as `StudentName` and `budget` as `DeptBudget`.
- Note that the `budget` attribute does not have any prefix because it occurs only in the `department` relation.

Output:

studentname	deptbudget
Brandt	50000

WHERE: AND / OR

Where: AND and OR

Example

- From the `instructor` and `department` relations in the figure, find out the names of all instructors whose department is 'Finance' or whose department is in any of the following buildings: 'Watson', 'Taylor'.

name
Srinivasan
Wu
Einstein
Gold

Where: AND and OR

Example

- From the instructor and department relations in the figure, find out the names of all instructors whose department is 'Finance' or whose department is in any of the following buildings: 'Watson', 'Taylor'.
- **Query:**

```
select name
from instructor I, department D
where D.dept_name = I.dept_name
  and (I.dept_name = 'Finance' or building in ('Watson','Taylor'));
```

name
Srinivasan
Wu
Einstein
Gold

Where: AND and OR

Example

- From the `instructor` and `department` relations in the figure, find out the names of all instructors whose department is 'Finance' or whose department is in any of the following buildings: 'Watson', 'Taylor'.
- **Query:**

```
select name
from instructor I, department D
where D.dept_name = I.dept_name
  and (I.dept_name = 'Finance' or building in ('Watson','Taylor'));
```

- **Output:**

name
Srinivasan
Wu
Einstein
Gold

Instructor and Department Relations

department

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

String

String Operations

Pattern Matching

- From the `course` relation in the figure, find the titles of all courses whose course id has three alphabets indicating the department.

title
Intro. to Biology
Genetics
Computational Biology
Investment Banking
World History

String Operations

Pattern Matching

- From the course relation in the figure, find the titles of all courses whose course id has three alphabets indicating the department.
- **Query:**

```
select title
from course
where course_id like '___-%';
```

title
Intro. to Biology
Genetics
Computational Biology
Investment Banking
World History

String Operations

Pattern Matching

- From the course relation in the figure, find the titles of all courses whose course id has three alphabets indicating the department.
- **Query:**

```
select title
from course
where course_id like '___-%';
```

- **Output:**

title
Intro. to Biology
Genetics
Computational Biology
Investment Banking
World History

String Operations

Pattern Matching

- From the course relation in the figure, find the titles of all courses whose course id has three alphabets indicating the department.
- **Query:**

```
select title
from course
where course_id like '___-%';
```

- **Output:**

title
Intro. to Biology
Genetics
Computational Biology
Investment Banking
World History

course relation

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

ORDER BY

Order By

Overview

- From the `student` relation in the figure, obtain the list of all students in alphabetic order of departments and within each department, in decreasing order of total credits.

Order By

Overview

- From the `student` relation in the figure, obtain the list of all students in alphabetic order of departments and within each department, in decreasing order of total credits.
- The list is first sorted in alphabetic order of `dept_name`.

Order By

Overview

- From the `student` relation in the figure, obtain the list of all students in alphabetic order of departments and within each department, in decreasing order of total credits.
- The list is first sorted in alphabetic order of `dept_name`.
- Within each dept, it is sorted in decreasing order of `tot_cred`.

Order By

Overview

- From the student relation in the figure, obtain the list of all students in alphabetic order of departments and within each department, in decreasing order of total credits.
- The list is first sorted in alphabetic order of dept_name.
- Within each dept, it is sorted in decreasing order of tot_cred.
- **Query:**

```
select name, dept_name, tot_cred
from student
order by dept_name ASC, tot_cred DESC;
```

Order By - student relation and output

student

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

Output

name	dept_name	tot_cred
Tanaka	Biology	120
Zhang	Comp. Sci.	102
Brown	Comp. Sci.	58
Williams	Comp. Sci.	54
Shankar	Comp. Sci.	32
Bourikas	Elec. Eng.	98
Aoi	Elec. Eng.	60
Chavez	Finance	110
Brandt	History	80
Sanchez	Music	38
Peltier	Physics	56
Levy	Physics	46
Snow	Physics	0

IN

In Operator

Overview

- From the `teaches` relation in the figure, find the IDs of all courses taught in the Fall or Spring of 2018.

In Operator

Overview

- From the `teaches` relation in the figure, find the IDs of all courses taught in the Fall or Spring of 2018.
- *Note: We can use `distinct` to remove duplicates.*

In Operator

Overview

- From the teaches relation in the figure, find the IDs of all courses taught in the Fall or Spring of 2018.
- *Note: We can use `distinct` to remove duplicates.*
- **Query:**

```
select course_id
from teaches
where semester in ('Fall','Spring') and year = 2018;
```

In Operator - teaches relation and output

teaches

ID	course_id	semester	year
10101	CS-101	Fall	2017
10101	CS-315	Spring	2018
10101	CS-347	Fall	2017
12121	FIN-201	Spring	2018
15151	MU-199	Spring	2018
22222	PHY-101	Fall	2017
32343	HIS-351	Spring	2018
45565	CS-101	Spring	2018
45565	CS-319	Spring	2018
76766	BIO-101	Summer	2017
76766	BIO-301	Summer	2018
83821	CS-190	Spring	2017
83821	CS-190	Spring	2017

Output

course_id
CS-315
FIN-201
MU-199
HIS-351
CS-101
CS-319
CS-319

Set

Set Operations: UNION

Overview

- For the same question in the previous slide, we can find the solution using `union` operator as follows.

Set Operations: UNION

Overview

- For the same question in the previous slide, we can find the solution using `union` operator as follows.
- **Query:**

```
select course_id from teaches
where semester='Fall' and year=2018

union

select course_id from teaches
where semester='Spring' and year=2018;
```

Set Operations: UNION

Overview

- For the same question in the previous slide, we can find the solution using `union` operator as follows.
- **Query:**

```
select course_id from teaches
where semester='Fall' and year=2018

union

select course_id from teaches
where semester='Spring' and year=2018;
```

- Note that `union` removes all duplicates. If we use `union all` instead of `union`, we get the same set of tuples as in previous slide.

Set Operations (2): INTERSECT

Example

- From the `instructor` relation, find the names of all instructors who taught in either the Computer Science department or the Finance department and whose salary is ¡ 80000.

Set Operations (2): INTERSECT

Example

- From the `instructor` relation, find the names of all instructors who taught in either the Computer Science department or the Finance department and whose salary is ¡ 80000.
- **Query:**

```
select name
from instructor
where dept_name in ('Comp. Sci.','Finance')

intersect

select name
from instructor
where salary < 80000;
```

Set Operations (2): INTERSECT

Example

- From the `instructor` relation, find the names of all instructors who taught in either the Computer Science department or the Finance department and whose salary is ¡ 80000.
- **Query:**

```
select name
from instructor
where dept_name in ('Comp. Sci.','Finance')

intersect

select name
from instructor
where salary < 80000;
```

- Note that the same can be achieved using the query:

```
select name from instructor
```

Set Operations (3): EXCEPT

Example

- From the `instructor` relation, find the names of all instructors who taught in either the Computer Science department or the Finance department and whose salary is either ≥ 90000 or ≤ 70000.

Set Operations (3): EXCEPT

Example

- From the `instructor` relation, find the names of all instructors who taught in either the Computer Science department or the Finance department and whose salary is either ≥ 90000 or ≤ 70000.
- **Query:**

```
select name
from instructor
where dept_name in ('Comp. Sci.','Finance')

except

select name
from instructor
where salary < 90000 and salary > 70000;
```

Set Operations (3): EXCEPT

Example

- From the `instructor` relation, find the names of all instructors who taught in either the Computer Science department or the Finance department and whose salary is either ≥ 90000 or ≤ 70000.
- **Query:**

```
select name
from instructor
where dept_name in ('Comp. Sci.','Finance')

except

select name
from instructor
where salary < 90000 and salary > 70000;
```

- Note that the same can be achieved using the query given below:

```
select name from instructor
where dept_name in ('Comp. Sci.', 'Finance')
```

Aggregation

Aggregate functions: AVG

Average

- From the `classroom` relation, find the names and the average capacity of each building whose average capacity is greater than 25.
- **Query:**

```
select building, avg(capacity)
from classroom
group by building
having avg(capacity) > 25;
```

Output:

building	avg
Taylor	70
Packard	500
Watson	40

Aggregate functions (2): MIN

Minimum

- From the `instructor` relation, find the least salary drawn by any instructor among all the instructors.

least_salary
40000

Aggregate functions (2): MIN

Minimum

- From the `instructor` relation, find the least salary drawn by any instructor among all the instructors.
- **Query:**

```
select min(salary) as least_salary
from instructor;
```

least_salary
40000

Aggregate functions (2): MIN

Minimum

- From the `instructor` relation, find the least salary drawn by any instructor among all the instructors.
- **Query:**

```
select min(salary) as least_salary
from instructor;
```

- **Output:**

least_salary
40000

Aggregate functions (3): MAX

Maximum

- From the `student` relation, find the maximum credits obtained by any student among all the students.

max_credits
120

Aggregate functions (3): MAX

Maximum

- From the student relation, find the maximum credits obtained by any student among all the students.
- **Query:**

```
select max(tot_cred) as max_credits
from student;
```

max_credits
120

Aggregate functions (3): MAX

Maximum

- From the student relation, find the maximum credits obtained by any student among all the students.
- **Query:**

```
select max(tot_cred) as max_credits
from student;
```

- **Output:**

max_credits
120

Aggregate functions (4): COUNT

Count

- From the `section` relation, find the number of courses run in each building.

building	course_count
Taylor	5
Packard	4
Painter	3
Watson	3

Aggregate functions (4): COUNT

Count

- From the section relation, find the number of courses run in each building.
- **Query:**

```
select building, count(course_id) as course_count
from section
group by building;
```

building	course_count
Taylor	5
Packard	4
Painter	3
Watson	3

Aggregate functions (4): COUNT

Count

- From the section relation, find the number of courses run in each building.
- **Query:**

```
select building, count(course_id) as course_count
from section
group by building;
```

- **Output:**

building	course_count
Taylor	5
Packard	4
Painter	3
Watson	3

Aggregate functions (5): SUM

Sum

- From the `course` relation, find the total credits offered by each department.

dept_name	sum_credits
Finance	3
History	3
Physics	4
Music	3
Comp. Sci.	17
Biology	11

Aggregate functions (5): SUM

Sum

- From the course relation, find the total credits offered by each department.
- **Query:**

```
select dept_name, sum(credits) as sum_credits
from course
group by dept_name;
```

dept_name	sum_credits
Finance	3
History	3
Physics	4
Music	3
Comp. Sci.	17
Biology	11

Aggregate functions (5): SUM

Sum

- From the course relation, find the total credits offered by each department.
- **Query:**

```
select dept_name, sum(credits) as sum_credits
from course
group by dept_name;
```

- **Output:**

dept_name	sum_credits
Finance	3
History	3
Physics	4
Music	3
Comp. Sci.	17
Biology	11

Module Summary

Summary

- SQL Examples have been practiced for

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select
  - Cartesian Product / as

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select
  - Cartesian Product / `as`
  - Where: `and` / `or`

Slides used in this presentation are borrowed from `http://db-book.com/` with kind permission of the authors.
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select
  - Cartesian Product / `as`
  - Where: `and` / `or`
  - String Matching

Slides used in this presentation are borrowed from `http://db-book.com/` with kind permission of the authors.
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select
  - Cartesian Product / `as`
  - Where: `and` / `or`
  - String Matching
  - `Order by`

Slides used in this presentation are borrowed from `http://db-book.com/` with kind permission of the authors.
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select
  - Cartesian Product / `as`
  - Where: `and` / `or`
  - String Matching
  - `Order by`
  - `in`

Slides used in this presentation are borrowed from `http://db-book.com/` with kind permission of the authors.
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select
  - Cartesian Product / `as`
  - Where: `and` / `or`
  - String Matching
  - `Order by`
  - `in`
  - Set Operations: `union`, `intersect`, `except`

Slides used in this presentation are borrowed from `http://db-book.com/` with kind permission of the authors.
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- SQL Examples have been practiced for
  - Select
  - Cartesian Product / `as`
  - Where: `and` / `or`
  - String Matching
  - `Order by`
  - `in`
  - Set Operations: `union`, `intersect`, `except`
  - Aggregate Functions: `avg`, `min`, `max`, `count`, `sum`

Slides used in this presentation are borrowed from `http://db-book.com/` with kind permission of the authors.
Edited and new slides are marked with 'PPD'.